Progetto Apollo (ingegneria del software)

Alessandro Lemme

Università degli studi di Brescia

29 settembre 2026

Sommario

1. Pattern GRASP
   - Information Expert
   - Pure Fabrication
2. Pattern SOLID
   - Open Closed
   - Principio di separazione modello-vista
3. Gang og Four
   - Strategy
   - Proxy
4. Testing
   - White box
5. Refactoring
   - Replace Conditional with Polymorphism

Information Expert

Qual è il principio base nell'assegnazione di responsabilità agli oggetti?

I vari engine hanno accesso a (parte) del database e soddisfano le richieste prendendo i dati a loro strettamente necessari e formulando una risposta - dunque la responsabilità di formulare una risposta è attribuita alla classe che ha la possibilità di reperire i dati necessari.

Pure Fabrication

Come assegnare le responsabilità quando usando Information Expert si violano Low Coupling e High Cohesion?

Tutte le classi del pacchetto Engine hanno l'unica cosa in comune di agire sul database: mentre, essendo simil RESTful API (invece che POST si usa Set*, PUT diventa Edit*, GET e DELETE sono Get* e Delete*) al posto di esser divise per tipologia di richiesta sono tutte nello stesso pacchetto ordinate alfabeticamente di modo che il sistemista possa al bisogno alterare le query modificando un solo pacchetto (seguendo anche il principio del minimo privilegio, così facendo al sistemista può essere revocato il permesso di modifica al di fuori del suo pacchetto di competenza).

Open Closed

Per estendere le funzioni del server bisogna:

- Creare un oggetto in `Request` che implementi `RequestInterface` e prenda i dati necessari a formulare la richiesta.
- Creare un metodo in `Client` che prenda i dati e costruisca l'oggetto `Request`.
- Creare un oggetto in `Engine` che implementi `EngineInterface` e soddisfi la richiesta.
- Creare un oggetto in `Reply` che implementi `ReplyInterface` che contenga i dati richiesti.

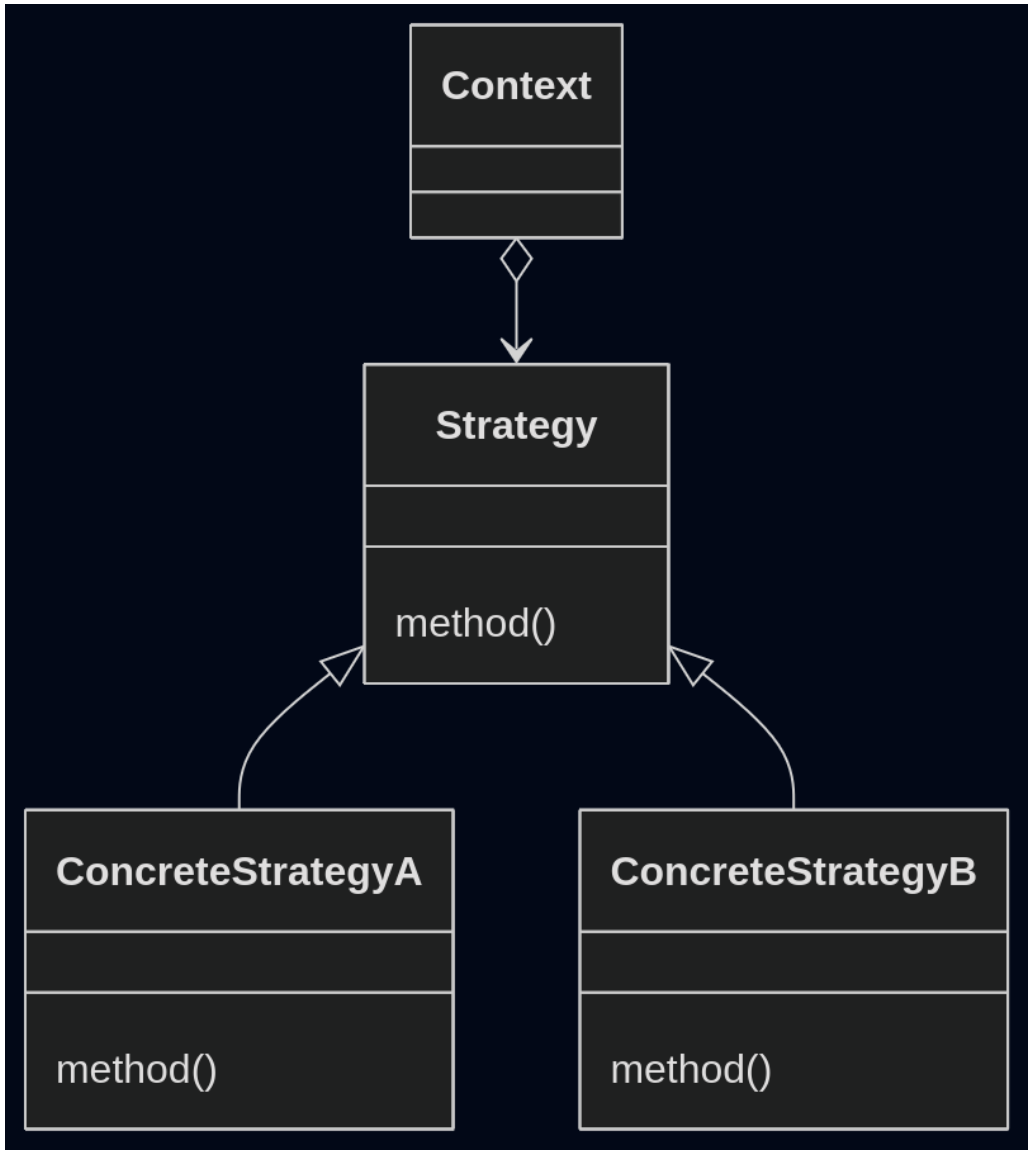
Principio di separazione modello-vista

I dati seguono una pipeline fortemente compartimentata - anche dovuto al modello architetturale Fat Client - l'interfaccia grafica dipende da una classe sottostante responsabile della creazione delle richieste (un oggetto JSON), la quale manda la richiesta al server che risponderà con un JSON che verrà "spacchettato" dal Client e che l'interfaccia grafica mostrerà all'utente.

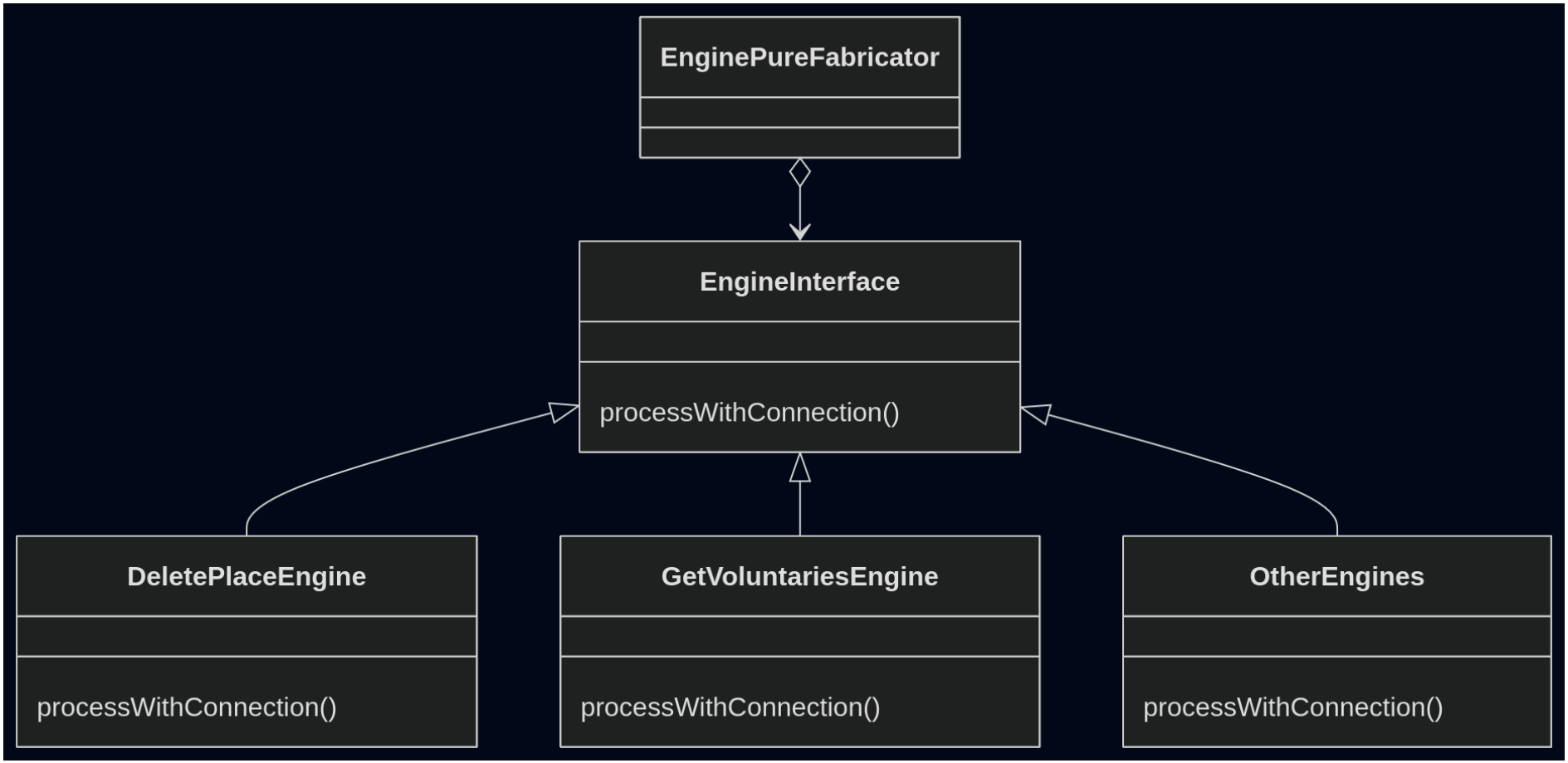
Strategy

- **Strategy**: `EngineInterface` è l'interfaccia a cui tutti i contenitori di algoritmi devono rifarsi per essere creati ed invocati da `Server`.
- **Context**: `EnginePureFabricator` è il context - contiene un riferimento ad un Engine, `EngineInterface` è l'oggetto strategy, dichiara un'interfaccia comune agli algoritmi supportati mentre i vari *Engine sono i concrete strategy dato che implementano l'algoritmo usando strategy.
- **ConcreteStrategy**: i vari engine contengono l'algoritmo per rispondere alle richieste.

Strategy generico



Strategy specifico



Proxy

Il database SQL di java.sql agisce da proxy, assieme ad H2

- remoto: qualora il DB fosse altrove l'interfaccia java.sql gestisce la connessione
- virtuale: H2 che implementa java.sql gestisce il caricamento in memoria dei dati del database
- protezione: l'accesso al database non è totalmente libero, serve una autenticazione
- intelligente: "consente l'esecuzione di attività aggiuntive quando si accede al soggetto" - ovvero le query

Testing white box

I test eseguiti su `GetPersonalDataEngine` sono tre per i due possibili metodi che potrebbero venir chiamati.

- `testPetitionerCanLogIn()` controlla solo che l'utente `Lancillotto.Benacense.99` possa connettersi con le sue credenziali corrette.
- `testPetitionerFailLogIn()` controlla solo che l'utente `Lancillotto.Benacense.99` non possa connettersi con le sue credenziali errate.
- `testProcessWithConnection()` controlla che l'utente ritornato contenga tutti i dati che dovrebbe.
- `testProcessWithConnectionShouldFail()` controlla che un utente inesistente non possa accedere a dati del database.

Replace Conditional with Polymorphism

Originalmente `Server` si occupava di scegliere quale oggetto `Engine` instanziare, tramite una lunga serie di condizioni. Adesso questa responsabilità gli è stata rimossa ed attribuita ad `EnginePureFabricatorContext` che contiene un dizionario con alle chiavi i vari enum di `ComunicationType` e con un riferimento nei valori agli oggetti di tipo `EngineInterface` corrispondenti.